Institución: Universidad Politécnica de Yucatán
Materia: Data Visualization (4th° Cuatrimestre)
Profesor: Jorge Javier Pedrozo Romero
Alumno: Yahel Cardenas Celis | **Grupo:** 4 B
Fecha: 2026-02-23

---

1. Intoduction

This report presents the development and results of Activity 8 of the Data Engineering and Artificial Intelligence course. The main objective of this activity was to implement complete CRUD operations (Create, Read, Update, and Delete) on relational databases, as well as to understand and apply database transactions following the ACID properties. Building upon the database connection fundamentals established in Activity 7, this workshop explored professional-level database operations that are essential in any modern data-driven application. The work environment consisted of two database engines running in Docker containers: MySQL (managing an inventory system called inventory_db) and PostgreSQL (managing a sales system called sales_db). Python scripts were developed for each operation, incorporating the libraries mysql-connector-python, psycopg2, pandas, and matplotlib to interact with the databases and generate visualizations of the results.

2. Environment Setup

The first step was to start the Docker containers from Activity 7. The command docker-compose up -d was executed, which started both database containers: postgres_container and mysql_container. Both containers were confirmed to be running with their corresponding ports mapped (3306 for MySQL and 5432 for PostgreSQL).

```
➜  ACT7 docker compose up -d
WARN[0000] /home/yahelcc/Data_visualization/ACT7/docker-compose.yml: the attribute `version` is obsolete, it will b
e ignored, please remove it to avoid potential confusion
[+] Running 0/1
 ⠿ Network act7_data_network  Creating                                                                    0.2s
[+] Running 3/3t of type `volume` should not define `bind` option
 ✔ Network act7_data_network     Created                                                                  0.2s
 ✔ Container postgres_container  Started                                                                  0.7s
 ✔ Container mysql_container     Started                                                                  0.6s
➜  ACT7
```

Figura 1: *Docker containers started successfully.*

During the MySQL database initialization, an initial access error (ERROR 1044 – Access Denied) was encountered when attempting to run the SQL script with the datauser account. This occurred because the user did not yet have permissions to create databases. The issue was resolved by connecting as root and granting the necessary privileges with the command GRANT ALL PRIVILEGES ON inventory_db.* TO 'datauser'@'%', followed by FLUSH PRIVILEGES.

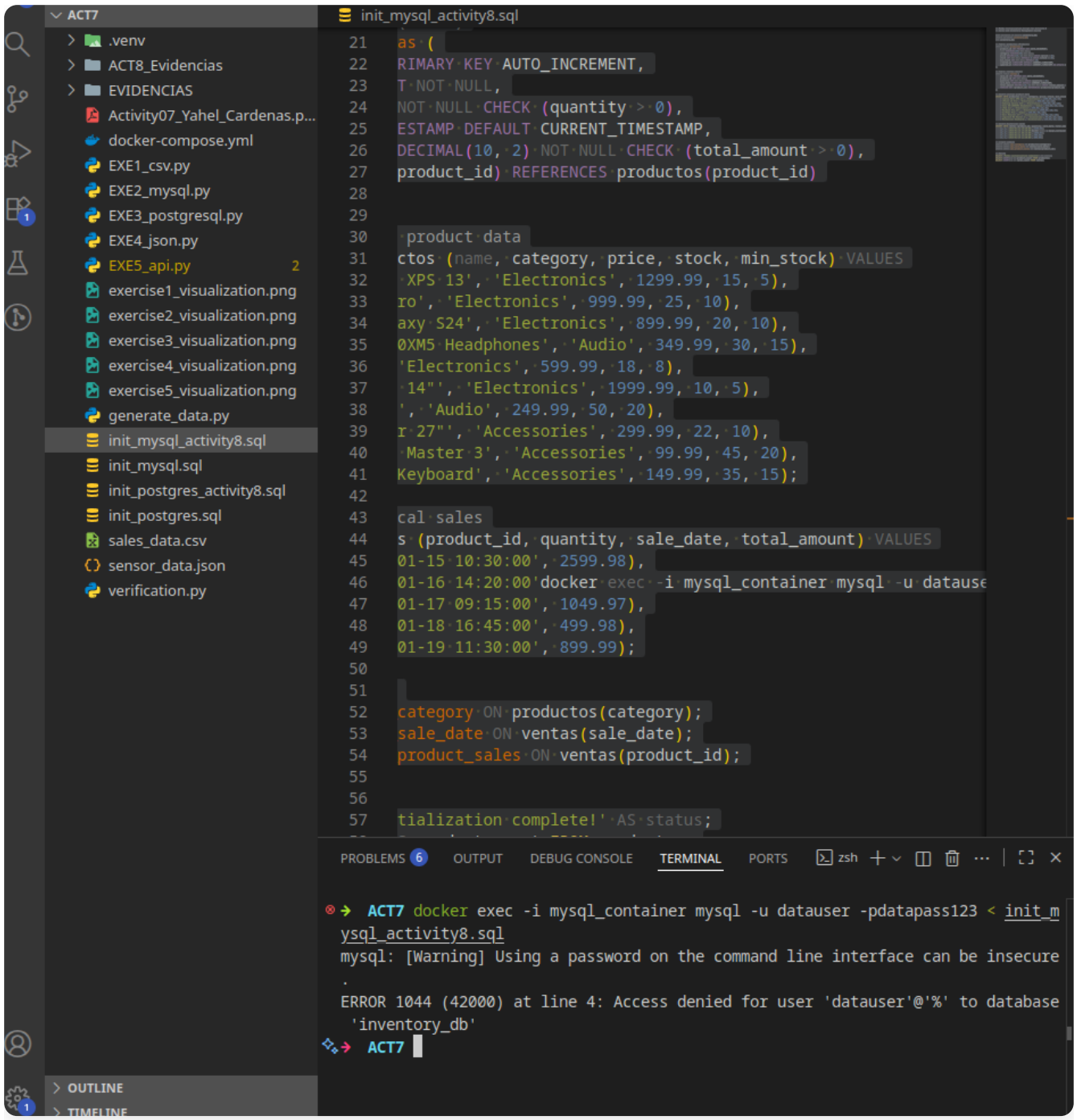

Figura 2: *initial access error when initializing MySQL and its resolution.*

After resolving the permissions issue, both databases were successfully initialized. The PostgreSQL initialization script confirmed 10 products and 5 orders in the sales_db schema. The MySQL database was populated with 10 products across three categories (Electronics, Audio, and Accessories) along with historical sales records.

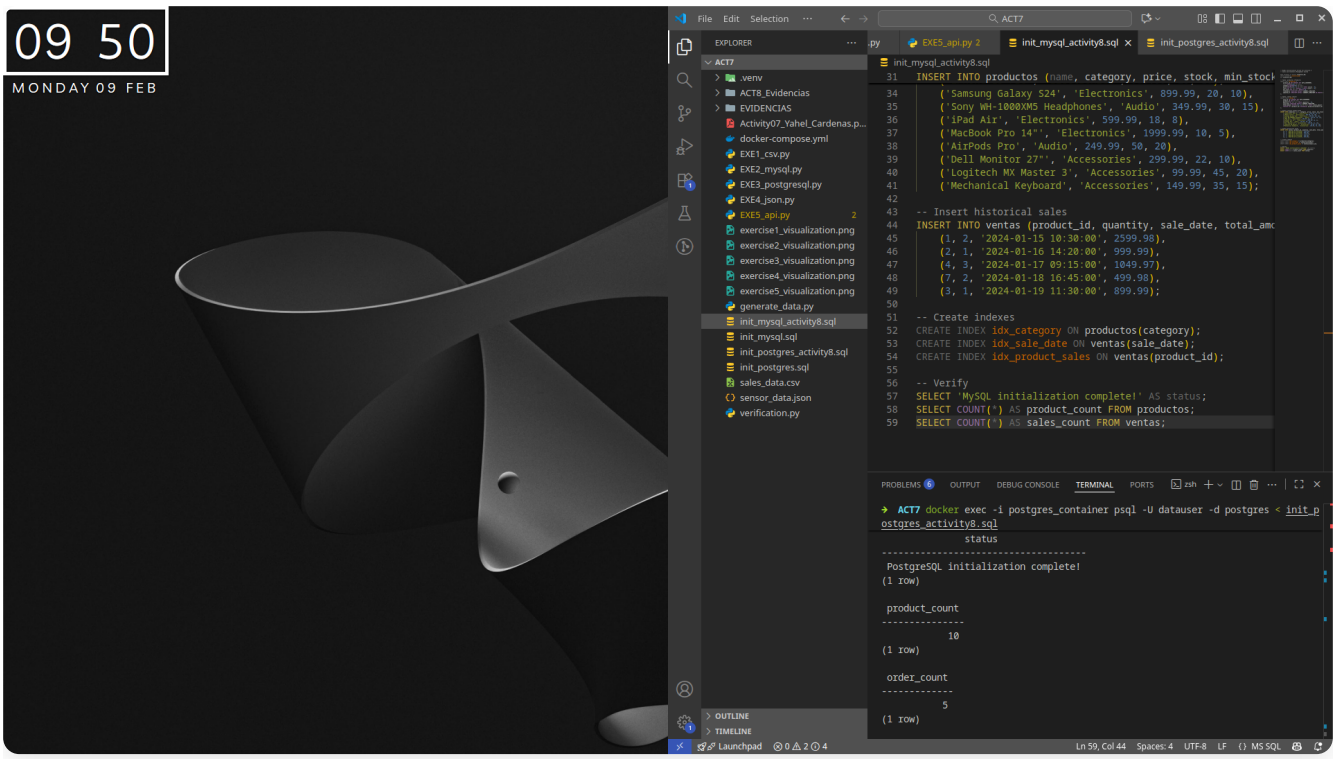

Figura 3: *Successful postgresSQL initialization (10 products, 5 orders).*

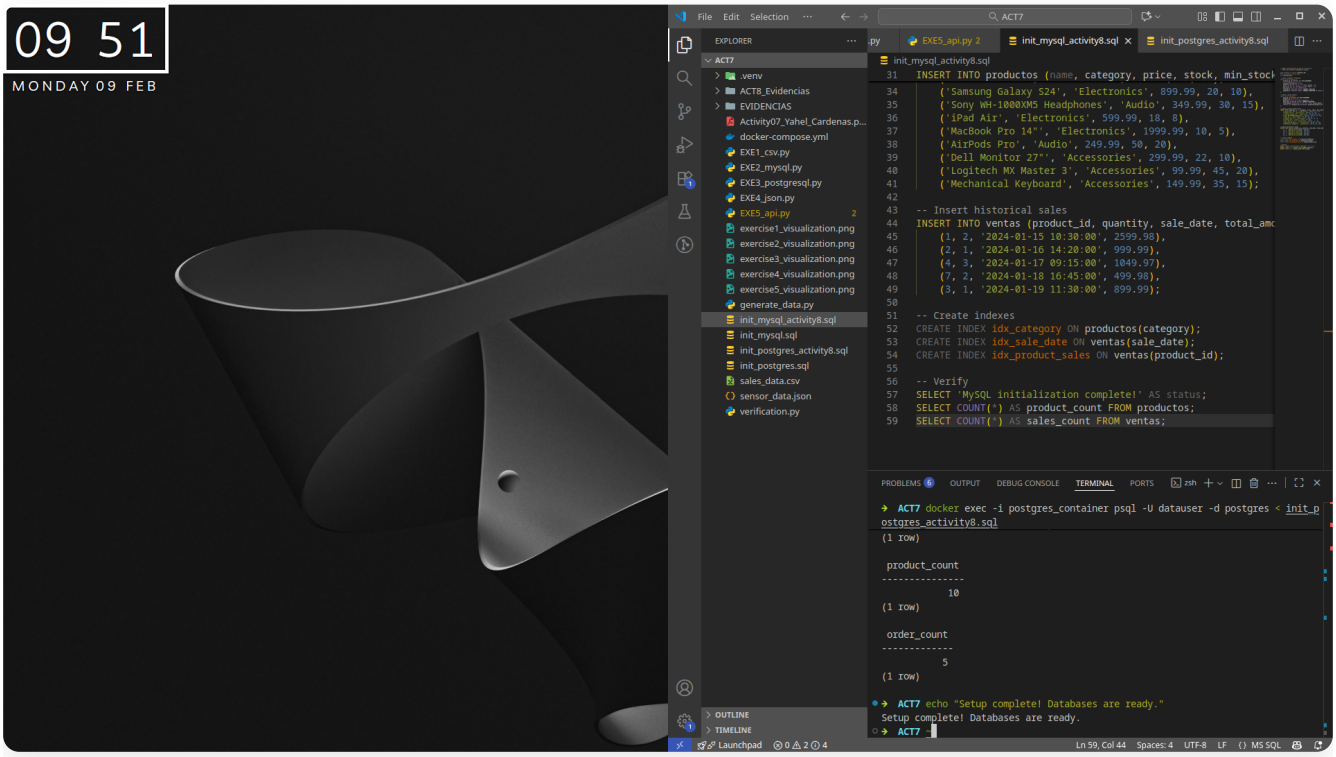

Figura 4: *Enviroment setup and verification complete.*

3. CRUD Operations

CRUD is an acronym that represents the four fundamental operations of any persistent storage system: Create, Read, Update, and Delete. These operations correspond directly to the SQL statements INSERT, SELECT, UPDATE, and DELETE, respectively. In this activity, each operation was implemented as a separate Python script targeting the MySQL inventory_db database.

3.1 CREATE – Inserting New Products

The CREATE operation uses SQL's INSERT INTO statement to add new records to a table. The script crud_create.py connected to the inventory_db database, which initially contained 10 products, and inserted three new items into the productos table:

id	Product	Category	price
11	google pixel 8	electronics	$699.99
12	bose quietcomfort earbuds	audio	$279.99
13	apple magic trackpad	accessories	$129.99

Tabla 1: *result of the INSERT INTO*

All three insertions were committed successfully, bringing the total product count from 10 to 13. The script verified this count after execution. An important consideration when using INSERT is ensuring data type compatibility and respecting constraints such as NOT NULL fields and CHECK conditions (e.g., price must be greater than 0, and stock cannot be negative). In real-world applications, the CREATE operation is used whenever new entities are registered in a system — for example, adding a new product to an e-commerce catalog or registering a new customer account.

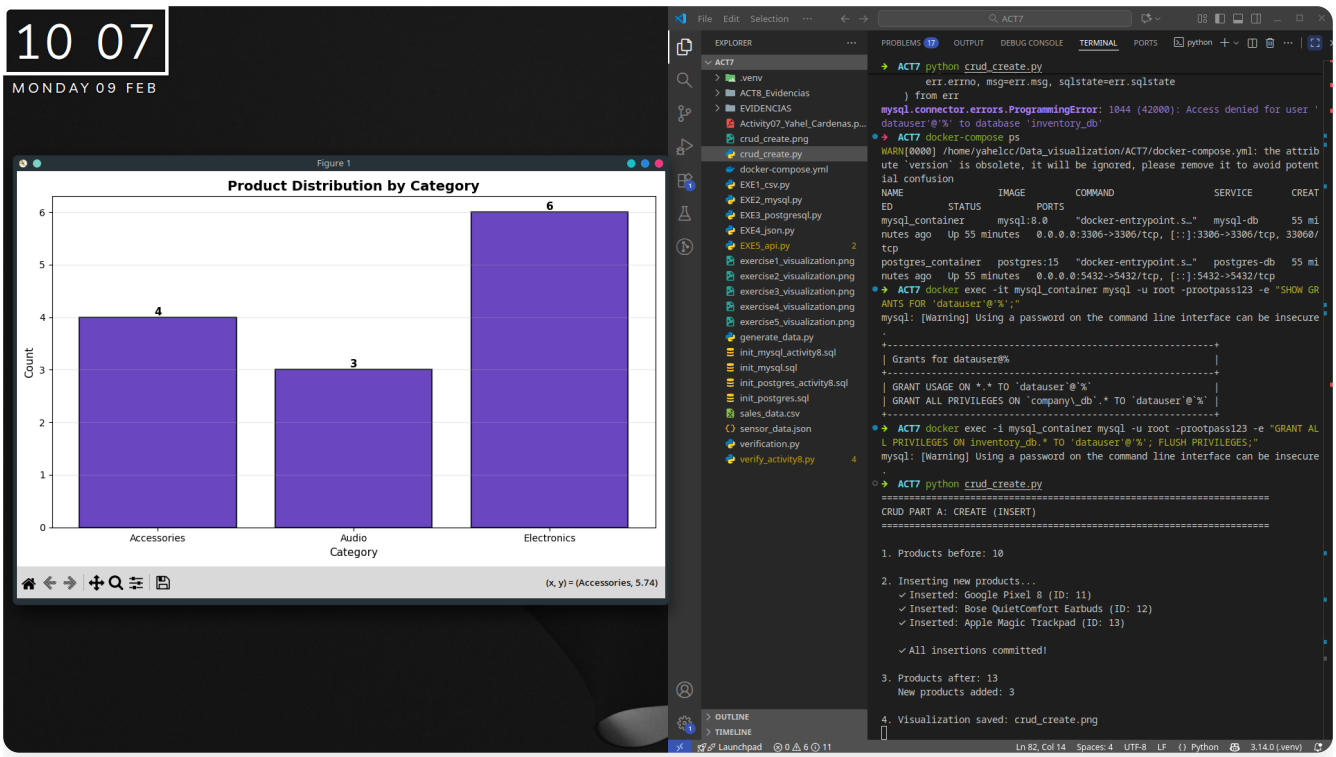

Figura 5: *Terminal output of crud_create.py showing 3 successful insertions.*

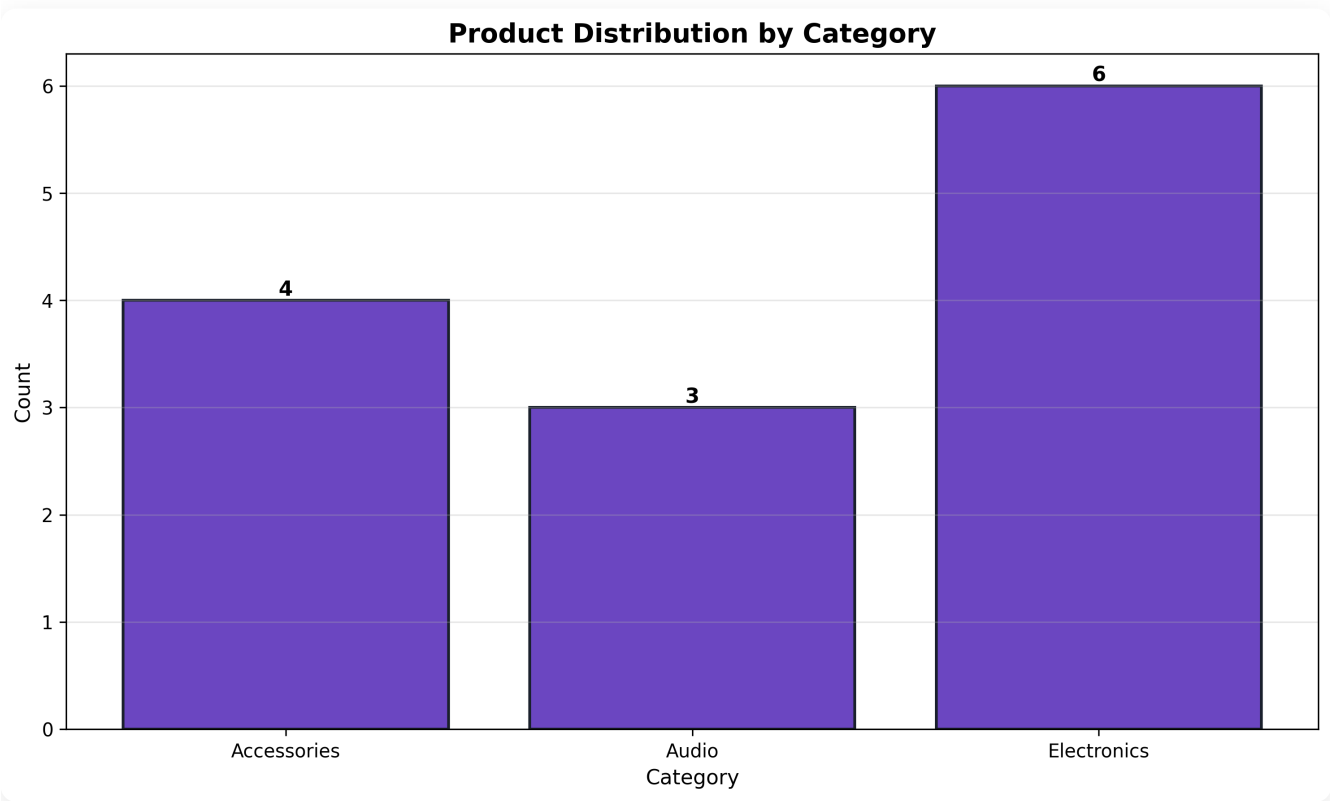

Figura 6: *Product distribution after CREATE (13 total products).*

3.2 READ – Querying the Database

The READ operation uses the SQL SELECT statement to retrieve data from one or more tables. The script crud_read.py executed a query that joined the productos and ventas tables to analyze sales performance by category. This type of JOIN operation is essential for extracting meaningful insights from relational data. The query retrieved historical sales data and aggregated it by product category. The results showed that the Electronics category generated the highest revenue at $4,499.96 across 3 sales involving 4 units, while the Audio category produced $1,549.95 from 2 sales totaling 5 units. Interestingly, Audio outsold Electronics in unit count, illustrating that revenue and volume are different metrics that must be analyzed together for informed business decisions.

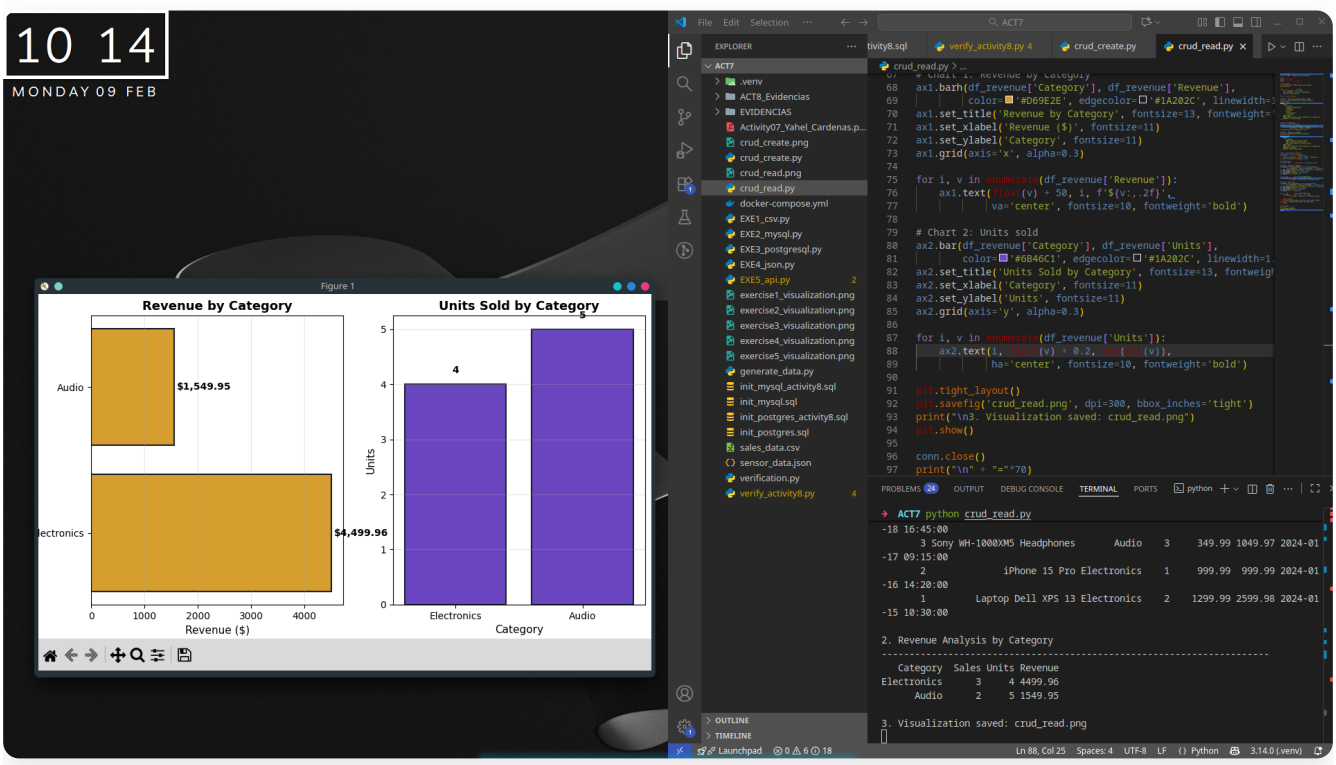

Figura 7: *Terminal output of crud_read.py showing revenue analysis.*

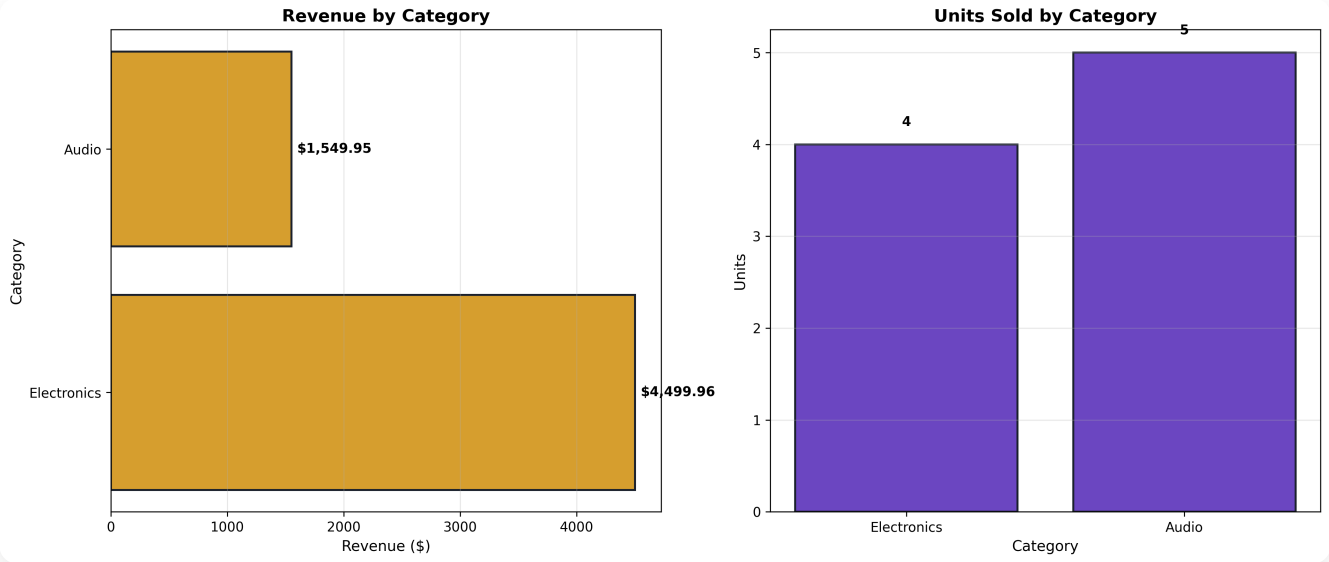

Figura 8: *Revenue by category and units sold by category charts.*

The READ operation is arguably the most frequently used CRUD operation in any application. Efficient SELECT queries rely on proper indexing — in this activity, indexes were created on the category and sale_date columns to accelerate lookups. Challenges in real-world READ operations include handling large datasets, optimizing JOIN performance, and designing queries that avoid full table scans.

3.3 UPDATE – Modifying Stock Levels

3.3 UPDATE – Modifying Stock Levels The UPDATE operation uses the SQL UPDATE ... SET statement to modify existing records. The script crud_update.py targeted products with critically low stock levels and applied restocking updates. The WHERE clause is absolutely critical in UPDATE operations: omitting it would update every row in the table, potentially corrupting the entire dataset. The following stock updates were applied and committed successfully:

product	stock before	stock after	change
laptop Dell xps 13	3	15	+12
iphone 15 pro	3	20	+17
samsung galaxy s24	3	20	+17

Tabla 2: *Table result of the UPDATE operation.*

After executing the updates, a verification query confirmed that the new stock values persisted correctly in the database. All three products had been below their minimum stock thresholds, and the UPDATE brought them back to acceptable levels. In production environments, UPDATE operations are often combined with business logic — for example, triggering a restock purchase order when inventory falls below a minimum threshold, exactly as demonstrated in this activity

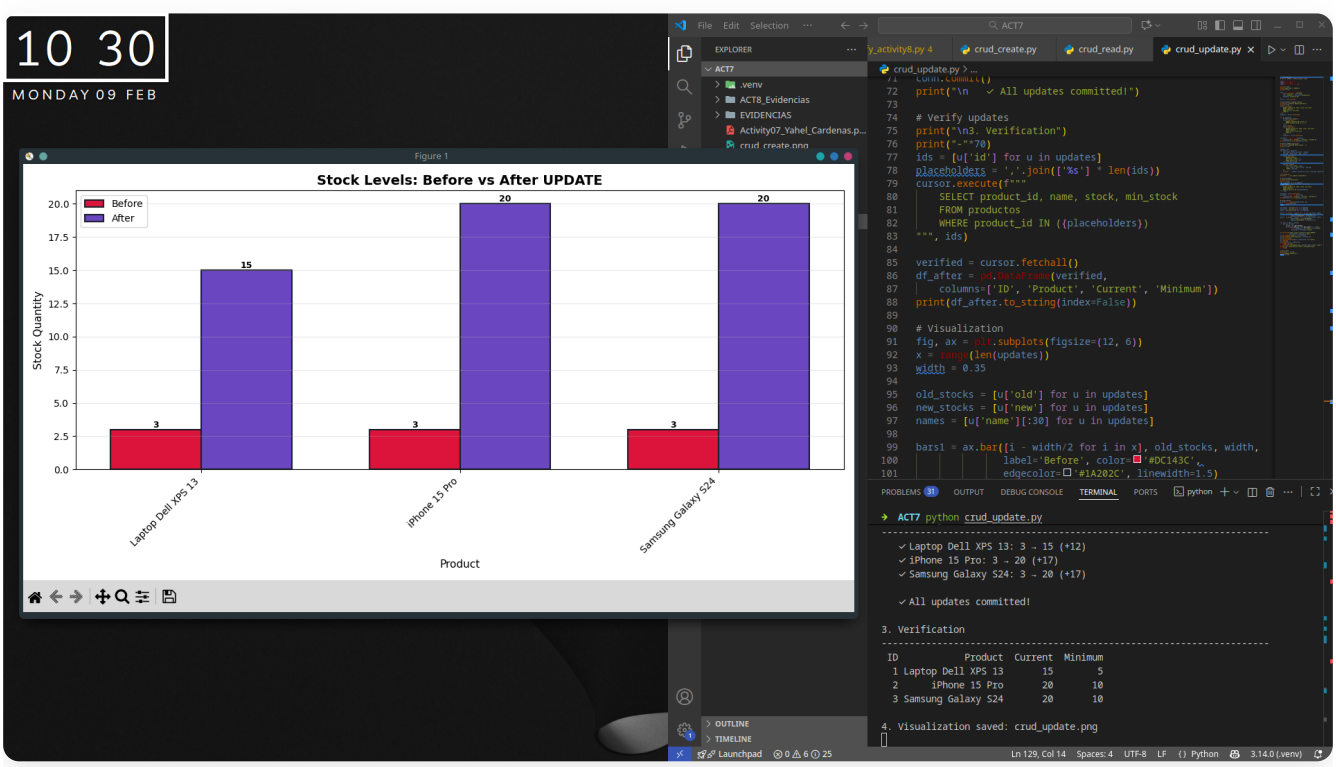

Figura 9: *Terminal output of crud_update.py confirming all stock updates*

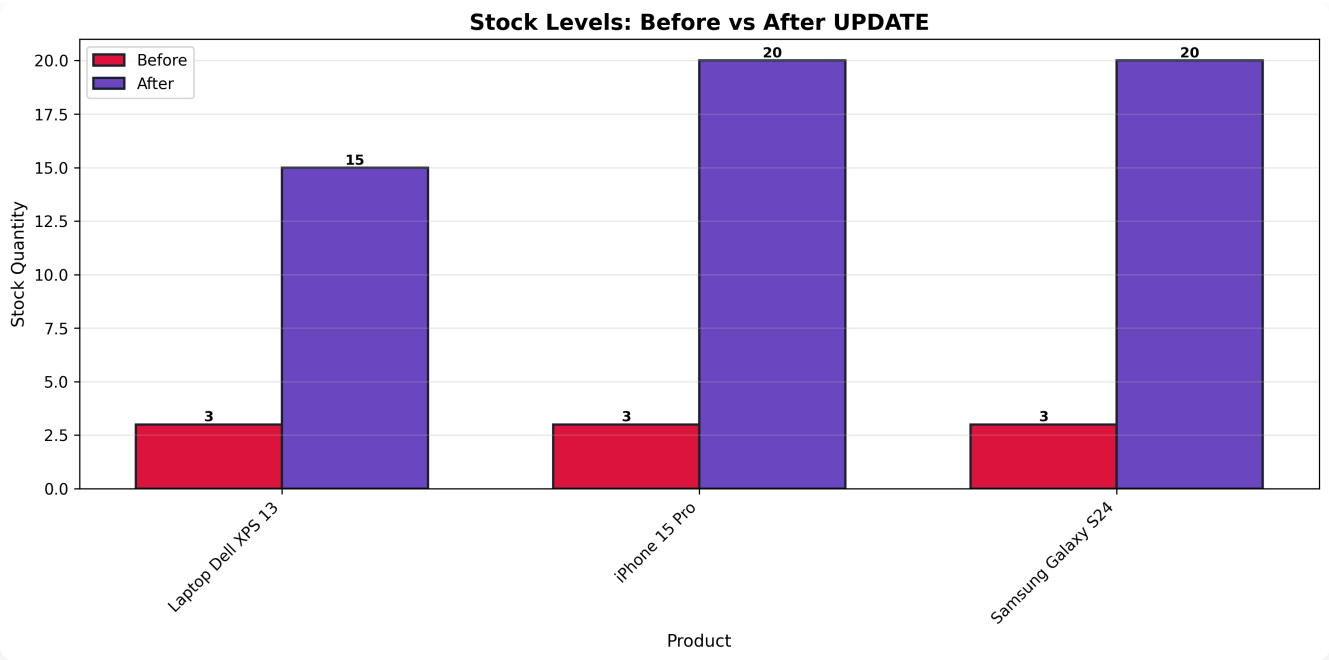

Figura 10: *Stock levels before and after the UPDATE operation.*

3.4 DELETE – Removing Discontinued Products

The DELETE operation removes rows from a table using the SQL DELETE FROM statement. Like UPDATE, the WHERE clause is critical — executing DELETE without it would erase every record in the table. The script crud_delete.py demonstrated safe deletion by first checking foreign key constraints before attempting to remove any product. The script first searched for products with stock below 5 units as candidates for deletion. It then verified whether any of those products had associated records in the ventas table — products referenced by sales records cannot be deleted due to the FOREIGN KEY constraint. Since no products with very low stock were free of sales references, the script created a temporary demo product (ID: 14) and immediately deleted it to demonstrate the DELETE functionality. The final inventory count remained at 13 products, distributed as: Accessories (4), Audio (3), and Electronics (6).

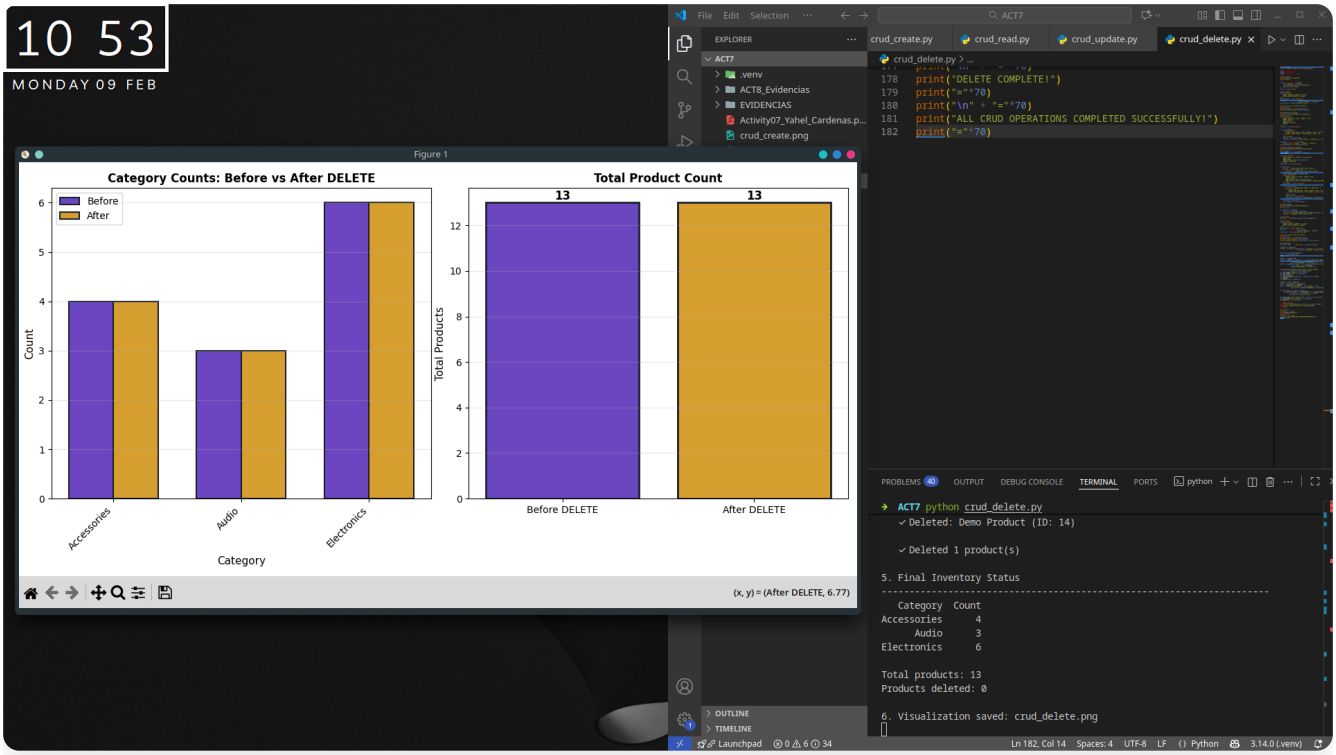

Figura 11: *Terminal output of crud_delete.py showing the deletion process.*

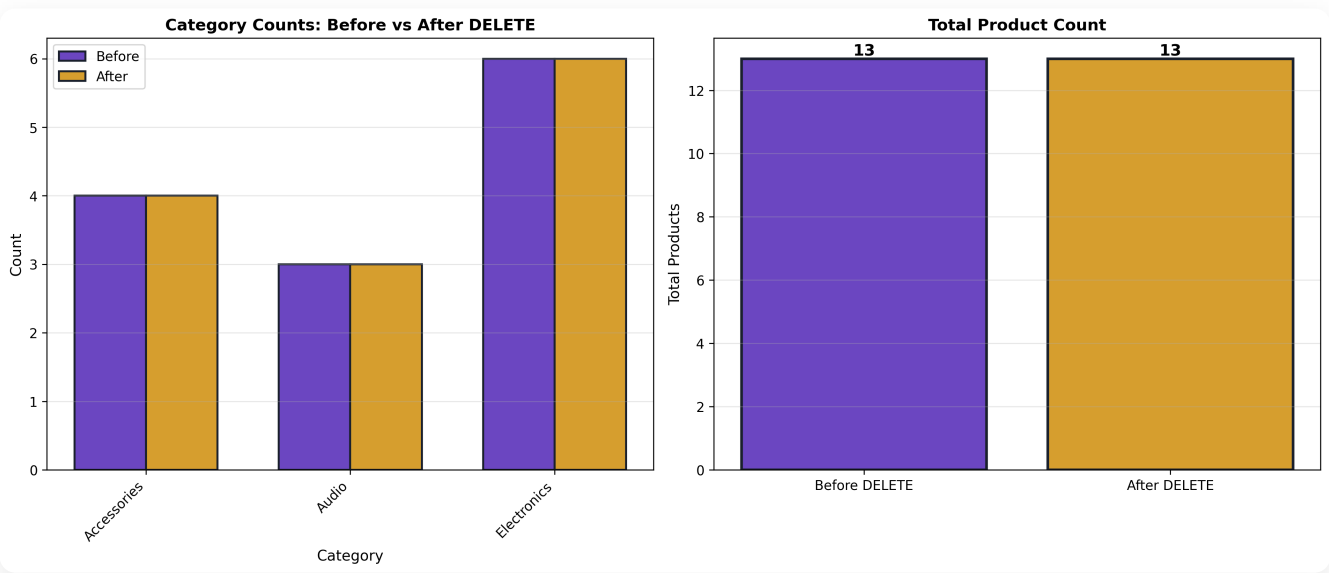

Figura 12: *Category counts before and after DELETE (totals remained at 13).*

4. ACID Properties

ACID is a set of properties that guarantee reliable processing of database transactions. The acronym stands for Atomicity, Consistency, Isolation, and Durability. These properties are essential for maintaining data integrity in any system where multiple operations must succeed or fail together — such as banking transfers, e-commerce checkouts, or inventory management. Both transactions implemented in this activity demonstrated all four ACID properties in practice.

4.1 Atomicity

All operations within a transaction are treated as a single indivisible unit. Either all operations succeed and are committed, or none of them take effect. In the COMMIT transaction, both the UPDATE to reduce stock and the INSERT into the sales table were committed together as one unit. In the ROLLBACK scenario, when the stock validation failed, no changes were applied at all — the database remained identical to its state before the transaction began.

4.2 Consistency

The database must transition from one valid state to another. All business rules and constraints must be satisfied before and after each transaction. In this activity, the CHECK constraint ensured that stock could never become negative, and the FOREIGN KEY relationship between ventas and productos was always maintained. The system enforces consistency at the database level, preventing logically invalid states.

4.3 Isolation

Concurrent transactions execute independently and do not interfere with each other. In the COMMIT transaction, the SELECT ... FOR UPDATE clause was used to lock the product row for the duration of the transaction. This prevented other users or processes from simultaneously modifying the same product's stock — a critical safeguard in any multi-user system where two customers might try to purchase the same last item in stock.

4.2 Durability

Once a transaction is committed, its changes are permanent and will persist even in the event of a system failure or power outage. After conn.commit() was called in the COMMIT transaction, the changes were written to the database's transaction log and to disk, guaranteeing they would survive any subsequent crash or restart of the database engine.

5. Database Transactions

A database transaction is a sequence of one or more SQL operations treated as a single logical unit of work. Transactions are controlled using three key commands: BEGIN (implicit when autocommit is disabled), COMMIT (to make changes permanent), and ROLLBACK (to undo all changes if an error occurs). This activity implemented both a successful transaction (COMMIT) and a failed transaction (ROLLBACK).

5.1 Transaction with COMMIT

The script transaction_commit.py simulated the purchase of 2 units of the iPhone 15 Pro by a customer. This transaction required coordinating two separate database operations atomically on the MySQL inventory_db:

• Operation 1: UPDATE productos — reduce stock by 2 units.

• Operation 2: INSERT INTO ventas — create a sales record.

The transaction was initiated with autocommit=False, which required an explicit COMMIT call to persist the changes. The product row was locked using SELECT ... FOR UPDATE to prevent concurrent modifications. After validating that sufficient stock was available (12 units on hand, 2 requested), both operations were executed and committed successfully. The stock of iPhone 15 Pro dropped from 12 to 10 units, and Sale record ID 10 was created with a total amount of $1,999.98.

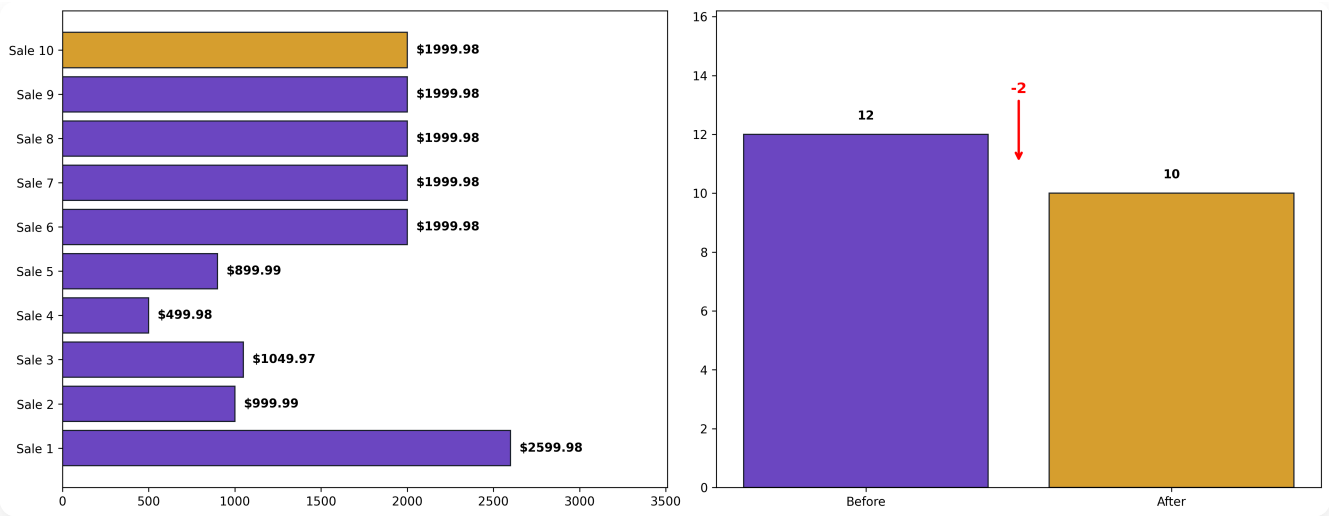

Figura 13: *Transaction COMMIT results: recent sales (Sale 10 highlighted in gold) and iPhone 15 Pro stock impact (12 → 10 units).*

The output confirmed all ACID properties: atomicity was demonstrated by the fact that both operations were committed together; consistency was verified by checking that the stock decrease matched the quantity sold; isolation was ensured by the row lock; and durability was confirmed by querying the database after COMMIT to verify the changes persisted.

5.2 Transaction with ROLLBACK

The script transaction_rollback.py demonstrated error handling in transactions by simulating a scenario in which a customer attempted to order 100 units of the 'Office Chair Pro' from the PostgreSQL sales_db, when only 40 units were available in stock. This scenario was designed to intentionally trigger a validation error to showcase the ROLLBACK mechanism.

The transaction followed these steps:

1. Captured the initial database state (stock = 40, order count recorded).

2. Locked the product row with SELECT ... FOR UPDATE.

3. Validated stock: 40 < 100 — insufficient stock detected, exception raised.

4. Executed conn.rollback() — all pending changes were discarded.

5. Verified the database state: stock remained at 40, order count unchanged.

The ROLLBACK mechanism is one of the most critical features of a transactional database. Without it, a partially executed transaction could leave the database in an inconsistent state — for example, a sale could be recorded without the corresponding stock reduction, or money could be debited from one bank account without being credited to another. Proper error handling with try-except-finally blocks in Python ensures that a ROLLBACK is always executed when an unexpected condition is detected during a transaction.

6. Conclusions

Activity 8 of the Data Visualization and Artificial Intelligence course introduced us to the fundamental concepts and best practices of CRUD operations and database transactions. What I found most fascinating was discovering that these practices did not originate with modern databases — they trace back to the 1970s with the Sequential Master Update algorithm, which performed a remarkably similar process using magnetic tapes: comparing records, verifying each change, and rewinding or canceling the new tape if anything did not match. Decades later, the core logic remains the same. Transactions prove to be extraordinarily useful in everyday computing, particularly in domains such as banking, logistics, and e-commerce. Understanding how they work — and how to handle them properly — is essential for any developer who aims to build reliable and efficient software that delivers real solutions to real clients.

Nombre del estudiante: Yahel Cardenas Celis

IA utilizada: Claude sonnet 4.6

Fecha de uso: 2026-02-23

Propósito: Writing assistance for the report

Prompt utilizado:

```
necesito ayuda con mi tarea. En los archivos html que te4 adjunte hay un tutorial el cual yo ya
```

```
realice, ahora necesito redactar un ensayo explicando todo lo que hicimos. Tu me ayudaras a
redaactar ese ensayo ya que soy muy malo escribiendo en ingles.

porfavor indicame si necesitas los outputs de los programas o si necesitas más informacion para
redactar el ensayo dime.
```

Archivos suministrados: HTML activity tutorials

Respuesta en crudo (raw):

```
https://claude.ai/share/537c6273-e64f-434d-9964-51f1dca1b321
```